善流投资_笔试_耿文博

题目1:

1. 定义一个先入后出的数据结构，实现这个类和它所需的成员函数;

2. 代码要有必要的注解(可以用中文);

```
#include <stdexcept>
#include <iostream>
#include <vector>
#include <assert.h>

template <typename T>
class Stack {
public:
    // 构造函数
    Stack(int size = 100) : m_size(size) {}

    // 析构函数
    ~Stack() {}

    // 入栈
    void push(T value) {
        // 判断栈是否已满
        if (m_data.size() == m_size) {
            // 栈已满，扩容
            m_size += 1;
            m_data.resize(m_size);
        }

        // 将元素压入栈顶
        m_data.push_back(value);
    }

    // 出栈
    T pop() {
        // 判断栈是否为空
        if (m_data.empty()) {
            // 栈为空，抛出异常
            throw std::out_of_range("Stack is empty");
        }

        // 返回栈顶元素
        T value = std::move(m_data.back());
        m_data.pop_back();
        return value;
    }

    // 获取栈顶元素
    T top() {
        // 判断栈是否为空
        if (m_data.empty()) {
            // 栈为空，抛出异常
            throw std::out_of_range("Stack is empty");
        }

        // 返回栈顶元素
        return m_data.back();
    }

    // 判断栈是否为空
    bool empty() {
        return m_data.empty();
    }

    // 获取栈的大小
```

```
    int size() {
        return (int)m_data.size();
    }

private:
    // 栈的大小
    int m_size;

    // 栈数据
    std::vector<T> m_data;
};
```

2. 编写单元测试，测试你的类的正确性，并简要说明测试目的或场景

上述代码实现了一个基于动态数组的栈(先入后出)，其测试目的或场景包括：

- 入栈操作：测试栈是否能够正确入栈元素。
- 出栈操作：测试栈是否能够正确出栈元素。
- 判断栈是否为空：测试栈是否能够正确判断栈是否为空。
- 空栈的出栈操作：测试空栈的出栈操作是否会抛出异常。
- 空栈的栈顶元素操作：测试空栈的栈顶元素操作是否会抛出异常。
- 栈的大小：测试栈的大小是否正确。

```
int main(int argc, const char * argv[]) {
    Stack<int> stack(10);

    // 入栈
    stack.push(1);
    stack.push(2);
    stack.push(3);

    // 出栈
    int value = stack.pop();
    std::cout << value << std::endl;
    value = stack.pop();
    std::cout << value << std::endl;
    value = stack.pop();
    std::cout << value << std::endl;

    // 判断栈是否为空
    std::cout << stack.empty() << std::endl;

    // 测试空栈的出栈操作
    try {
        stack.pop();
    } catch (const std::out_of_range& e) {
        std::cout << "出栈空栈，抛出异常：" << e.what() << std::endl;
    }

    // 测试获取栈顶元素的空栈操作
    try {
        stack.top();
    } catch (const std::out_of_range& e) {
        std::cout << "获取空栈的栈顶元素，抛出异常：" << e.what() << std::endl;
    }

    // 测试判断栈是否为空的空栈操作
    assert(stack.empty());

    // 测试栈的大小
    assert(stack.size() == 0);

    std::cout << "Hello, World!\n";
    return 0;
}
```

3. 分析你的算法的时间和空间复杂度

Time Complexity:

1. **push()：**如果栈的大小小于等于 `m_size`，则插入操作是 O(1) 的。如果栈的大小达到 `m_size`，则需要进行扩容，`resize` 操作会花费 O(n) 的时间。
2. **pop()：**pop 操作的时间复杂度为 O(1)，因为 vector 的 pop_back 操作是常数时间。
3. **top()：**top 操作的时间复杂度为 O(1) → 返回栈顶元素。
4. **empty()：**empty 操作的时间复杂度为 O(1) → 栈是否为空。
5. **size()：**size 操作的时间复杂度为 O(1) → 返回栈的大小。

Space Complexity: 主要取决于栈中存储的元素数量，即 vector 的大小 → O(n)。在最坏情况下，如果进行了多次扩容，空间复杂度为 O(n)，n 为栈的大小。

题目2:

1. 检查代码是否有错误，如是，请进行修改;

1. **潜在的内存泄漏** `find_and_erase0()` 用 `delete ptr` 从动态分配的数组中删除单个元素。但它只删除指针本身，但不删除指针指向的内存 → 导致内存泄漏。
2. **删除之后的无效访问** `find_and_erase1()` 用 `erase` 方法会使得迭代器会失效，所以要更新迭代器。
3. **不必要的复制构造函数和赋值运算符** copy constructor 还有 operator，只进行了 shadow copy 而不是 deep copy 所以只是复制内部数据结构的指针，可能会导致悬挂指针问题。

修改：

1. `vector` 代替原始指针来存储 `_data0` 因为 `vector` 提供了内置的内存管理功能，可以更加安全地删除元素。
2. 使用标准库中 `remove_if()` 来替代 `erase()` 删除元素。
3. 将 copy constructor 和 `operator=` 改为 deep copy 分配新内存并复制元素。

```
#include <vector>

class Foo {
public:
    // constructor
    Foo(int size)
        : _data0(size),
          _data1(size)
    {
        // 使用 vector 的构造函数来分配内存
        _data0 = std::vector<int>(size);
        _data1 = std::vector<int>(size);
    }

    // 避免浅拷贝
    Foo(const Foo &other) = delete;
    Foo &operator=(const Foo &other) = delete;

    // find all occurrences of @val and erase if there is any
    void find_and_erase0(int val)
    {
        // 使用 remove_if() 来删除元素
        auto it = std::remove_if(_data0.begin(), _data0.end(), [val](int element) { return element == val; });
        _data0.erase(it, _data0.end());
    }

    // find all occurrences of @val and erase if there is any
    void find_and_erase1(int val)
    {
        _data1.erase(std::remove_if(_data1.begin(), _data1.end(), [val](int element) { return element == val; }), _data1.end());
```

```
    }

private:
    std::vector<int> _data0;
    std::vector<int> _data1;
};
```

2. 分析各个函数的复杂度，如果函数可以进行优化，请对该函数进行重构。

1. 构造函数的复杂度为 O(n)，其中 n 是数组的大小。这是因为构造函数需要分配 n 个元素的内存。
2. copy constructor 和 `operator=` 为 O(n)，其中 n 是数组的大小因为这两个函数需要复制数组中的所有元素。
3. `find_and_erase0()` 函数的复杂度为 O(n^2)，其中 n 是数组的大小。由于`remove_if()` 的复杂度为 O(n)，并且需要遍历整个数组两次：查找要删除的元素 + 删除元素。
4. `find_and_erase1()` 函数的复杂度为 O(n)，由于`remove_if()` 的复杂度为 O(n)。

优化: 可以使用 `unique_ptr` 或 `shared_ptr` 来管理 `_data0` 和 `_data1` 的指针避免内存泄漏。

```
#include <vector>
#include <memory>

class Foo {
public:
    // constructor
    Foo(int size)
        : _data0(std::make_unique<std::vector<int>>(size)),
          _data1(std::make_unique<std::vector<int>>(size))
    {}

    Foo(const Foo &other) : Foo(static_cast<int>(other._data0->size())) { *_data0 = *other._data0; }
    Foo &operator=(const Foo &other) { *_data0 = *other._data0; return *this; }

    // find all occurrences of @val and erase if there is any
    void find_and_erase0(int val)
    {
        auto it = std::remove_if(_data0->begin(), _data0->end(), [val](int element) { return element == val; });
        _data0->erase(it, _data0->end());
    }

    // find all occurrences of @val and erase if there is any
    void find_and_erase1(int val)
    {
        _data1->erase(std::remove_if(_data1->begin(), _data1->end(), [val](int element) { return element == val; }), _data1->end());
    }

private:
    // Unique pointers 动态管理 vector
    std::unique_ptr<std::vector<int>> _data0;
    std::unique_ptr<std::vector<int>> _data1;
};
```

题目3:

1. 使用你认为最高效的算法，实现一个函数find_common_elements，它的输入是这k个数组，输出是一个有序数组，按序包含所有在这k个数组中都出现的元素;
2. 代码要有必要的注解(可以用中文);

Brute Force: 遍历 input，对所有数组的没给元素进行查找，最后对数组进行排序，找相同的元素 → O(n^k + nlogn) n 为所有输入数组中元素的总数。

优化: 两两进行查重，每次只保留重复的那部分，有重复的那部分再跟之后的数组进行比较，检查是否有相同的元素 ⇒ k 个数组都出现的元素。

- O(n*m + klog(k))，n 为数组的总长度，m 为数组的个数，k 为答案数组的长度。

```
//假设输入都是整型的情况下
vector<int> find_common_elements(const vector<vector<int>>& arrays) {
    if (arrays.size() < 2) return {};
    unordered_map<int, int> count;

    // 统计第一个数组中的元素
    for (int num : arrays[0]) count[num] = 1; // 标记出现过


    // 遍历剩余数组，更新标记结果，两两比较
    for (int i = 1; i < arrays.size(); ++i) {
        unordered_map<int, int> currentCount;

        // 查找当前数组中的元素
        for (int num : arrays[i]) currentCount[num] = 1;

        // 更新 count 中的标记结果，保留重复部分
        unordered_map<int, int> newCount;
        for (auto& [element, visited]: count) {
            if (currentCount.count(element))
                newCount[element] = 1;

        }
        count = newCount;
    }

    vector<int> ans;
    for (auto& [element, _] : count)
        ans.push_back(element);

    sort(ans.begin(), ans.end()); // 排序保证答案数组有序
    return ans;
}
```

3. 我们在下面列出了几个测试样例，请再添加至少三个你自己编写的测试例子，并简要说明测试目的或场景;

假设输入都是整型，且不考虑大数和超出整型表示范围，测试用例如下：

Edge Cases: max and min value in the vector {INT_MIN, INT_MAX} {INT_MIN, INT_MAX} {1, INT_MAX} ⇒ {INT_MAX}

Functional Cases: {{1, 1, 1}, {1, 1}, {1, 1, 1, 1}} ⇒ {1}

Special Cases: k 个数组都为空

```
int main(int argc, const char * argv[]) {
    // insert code here...
    vector<vector<int>> A1 = {{-1, 2, 7}, {2, 3, 7, 8}, {-3, 0, 1, 2, 4}, {1, 2, 3}};
    vector<int> result1 = find_common_elements(A1);
    // 输出结果 [2]
    for (int elem : result1)
        cout << elem << " ";
    cout << endl;

    vector<vector<int>> A2 = {{11}, {0, 2, 4, 6, 8, 10, 12, 14, 16}};
    vector<int> result2 = find_common_elements(A2);
    // 输出结果 []
    for (int elem : result2)
        cout << elem << " ";
    cout << endl;

    vector<vector<int>> A3 = {{1, 3, 5}, {1, 3, 5}, {1, 3, 5}};
    vector<int> result3 = find_common_elements(A3);
    // 输出结果 [1, 3, 5]
    for (int elem : result3)
        cout << elem << " ";
    cout << endl;
```

```
    vector<vector<int>> A4 = {{-1, -2, -3}, {-2, -3, -4}, {-3, -4, -5}};
    vector<int> result4 = find_common_elements(A4);
    // 三个数组都只包含一个元素，期望结果是 [-3]
    for (int elem : result4)
        cout << elem << " ";
    cout << endl;

    vector<vector<int>> A5 = {{}, {}};
    vector<int> result5 = find_common_elements(A5);
    // 所有数组都为空，期望结果是 []
    for (int elem : result5)
        cout << elem << " ";
    cout << endl;

    vector<vector<int>> A6 = {{1, 1, 1}, {1, 1}, {1, 1, 1, 1}};
    vector<int> result6 = find_common_elements(A6);
    // 三个数组相同但长度不同，期望结果是 [1]
    for (int elem : result6)
        cout << elem << " ";
    cout << endl;

    vector<vector<int>> A7 = {{1, 1, 1, 1}, {2, 3}, {5, 6}};
    vector<int> result7 = find_common_elements(A7);
    // 三个数组相同但长度不同，期望结果是 []
    for (int elem : result7)
        cout << elem << " ";
    cout << endl;

    vector<vector<int>> A8 = {{2, 2}, {2, 3}, {5, 6}};
    vector<int> result8 = find_common_elements(A8);
    //只有两个数组有相同元素，期望结果是 []
    for (int elem : result8)
        cout << elem << " ";
    cout << endl;

    vector<vector<int>> A9 = {{1, 2}};
    vector<int> result9 = find_common_elements(A9);
    // 只有一个数组
    for (int elem : result9)
        cout << elem << " ";
    cout << endl;

    vector<vector<int>> A10 = {{INT_MIN, INT_MAX, INT_MIN}, {INT_MAX, 1}, {INT_MAX, INT_MIN}};
    vector<int> result10 = find_common_elements(A10);
    // MIN MAX value, 期望结果是 [2147483647]
    for (int elem : result10)
        cout << elem << " ";
    cout << endl;

    return 0;
}
```

4. 分析你的算法的时间和空间复杂度;

Time Complexity: 在最坏情况下，需要遍历所有数组的所有元素所以是 O(n*m + klog(k)) n 为数组的总长度，m 为数组的个数，k 为答案数组的长度。

Space Complexity: O(n) 在不考虑返回值的情况下，使用了一个哈希表来存储元素的出现情况，n 为数组的总长度。在最坏情况下，所有元素都不重复，哈希表的大小将达到数组的总长度。